Fiche de Révision : Écosystème Python

Python Fondamental, NumPy, Matplotlib & Pandas — Fonctions & Méthodes Essentielles

1. Python Fondamental (Structures & Outils)

Fonction / Structure	Description & Usage
`list`, `tuple`, `dict`, `set`	Structures de base. **list** (mutable), **tuple** (immuable), **dict** (paires clé-valeur), **set** (éléments uniques, tests d'appartenance ultra-rapides en $O(1)$).
`.append(x)` / `.pop()`	Ajoute un élément à la fin d'une liste / retire et renvoie le dernier élément. Utilisé pour implémenter une pile (LIFO).
`zip(l1, l2)`	Parcourt plusieurs itérables en parallèle. Idéal pour aligner des clés et des valeurs.
`time.perf_counter()`	Chronométrage précis pour mesurer la complexité et les temps d'exécution algorithmiques.
`collections.deque`	File optimisée (double-ended queue), indispensable pour les parcours en largeur (BFS) dans les graphes.
`try: ... except: ...`	Gestion des erreurs (ex: intercepter une `ValueError`, `TypeError`, ou `KeyError`).

2. NumPy (Calcul Scientifique & Vectorisation)

Création & Inspection

Fonction	Description
`np.array(liste)`	Convertit une liste Python en tableau multidimensionnel optimisé (`ndarray`).
`np.zeros(n)` / `np.ones(n)`	Crée un tableau rempli de 0 ou de 1. Pratique pour initialiser des tampons.
`np.empty_like(arr)`	Crée un tableau non initialisé de la même forme que `arr`.
`np.arange(début, fin, pas)`	Génère un axe de valeurs avec un pas régulier (la **fin est exclue**).
`np.linspace(début, fin, n)`	Génère `n` valeurs régulièrement espacées (la **fin est incluse**).
`np.vstack((a, b))`	Empile verticalement plusieurs tableaux pour former une matrice.
`.shape`, `.ndim`, `.dtype`, `.size`	Attributs : dimensions (lignes, colonnes), nombre d'axes, type de données, et nombre total d'éléments.
`.reshape(lignes, cols)`	Réorganise la distribution des données sans les recopier.

Mathématiques, Réductions & Vérifications

Fonction	Description

`np.mean()`, `np.std()`, `np.min()`, `np.max()`	Statistiques descriptives. Accepte l'argument `axis=0` (par colonne) ou `axis=1` (par ligne).
`np.argmax(arr)`	Renvoie l'indice de la valeur maximale d'un tableau.
`np.sqrt()`, `np.exp()`, `np.log()`	Fonctions mathématiques vectorisées (racine, exponentielle, logarithme népérien).
`np.sin()`, `np.cos()`, `np.deg2rad()`	Fonctions trigonométriques (travaillent en radians).
`np.trapezoid(y, x)`	Calcule l'intégrale numérique d'une courbe via la méthode des trapèzes.
`np.polyfit(x, y, deg)`	Régression polynomiale. Avec `deg=1`, renvoie la pente et l'ordonnée à l'origine d'une droite d'ajustement.
`np.isclose()` / `np.allclose()`	Compare des nombres flottants avec une tolérance numérique (indispensable pour éviter les erreurs d'arrondi).

3. Matplotlib (Visualisation & Esprit Critique)

Fonction	Description
`fig, ax = plt.subplots(figsize=...)`	Crée la figure (le cadre) et l'axe (la zone de tracé). **Toujours privilégier cette API orientée objet.**
`ax.plot(x, y, label='...', marker='o')`	Trace une courbe continue ou une série temporelle ordonnée.
`ax.scatter(x, y)`	Trace un nuage de points pour évaluer l'association ou la corrélation entre deux variables.
`ax.bar(catégories, valeurs)`	Diagramme en barres pour comparer des catégories discrètes sans ordre inhérent.
`ax.hist(x, bins=10)`	Histogramme pour visualiser la distribution statistique d'une variable continue.
`ax.set_title()`, `ax.set_xlabel()`	Définit le titre du graphique et les labels des axes. **Ne jamais oublier les unités.**
`ax.set_xlim()`, `ax.set_ylim()`	Contrôle l'échelle de l'axe (utile pour éviter les biais visuels des axes tronqués).
`ax.grid(True)`, `ax.legend()`	Ajoute une grille de lecture légère et affiche la légende (nécessite l'argument `label` dans les tracés).
`fig.tight_layout()`	Ajuste automatiquement les marges pour éviter le rognage du texte ou des axes.
`fig.savefig('nom.png', dpi=150)`	Exporte le graphique en fichier image haute résolution.

4. Pandas (Analyse & Nettoyage de Données)

Chargement & Inspection

Fonction / Attribut	Description
`pd.read_csv('fichier.csv')`	Importe un fichier CSV dans un objet `DataFrame`.
`df.head()` / `df.tail()`	Affiche les premières ou dernières lignes pour contrôler l'importation.
`df.shape`, `df.dtypes`	Renvoie les dimensions (lignes, colonnes) et le type de données de chaque colonne.
`df.isna().sum()`	Compte le nombre de valeurs manquantes (`NaN`) par colonne.
`df.duplicated().sum()`	Compte le nombre de lignes exactement identiques.

Sélection, Nettoyage & Transformation

Fonction	Description
`df['colonne']`	Sélectionne une colonne (renvoie un objet `Series`).

`df.loc[masque, colonnes]`	Filtre et sélectionne les données via un masque booléen (ex: `df['âge'] > 18`) et des noms de colonnes.
`df.sort_values(by='col', ascending=False)`	Trie le DataFrame selon les valeurs d'une ou plusieurs colonnes.
`df.rename(columns={'A': 'B'})`	Renomme les colonnes (souvent utilisé pour uniformiser et standardiser les noms).
`df.replace(vieille_valeur, np.nan)`	Remplace des valeurs aberrantes ou spécifiques (souvent pour injecter proprement des NaN).
`pd.to_datetime(df['date'])`	Convertit une colonne de texte en type `datetime` pour l'analyse temporelle.
`df['date'].dt.hour`	Extrait une composante spécifique du temps (heure, mois, jour).

Agrégation & Croisement

Fonction	Description
`df['col'].value_counts(normalize=True)`	Compte les occurrences de chaque modalité. `normalize=True` donne les proportions.
`df.groupby(['catégorie']).agg(...)`	Regroupe les lignes partageant une même valeur, puis applique des fonctions d'agrégation (ex: `mean`, `size`, `sum`).
`df.reset_index()`	Remet à plat l'index généré par `groupby` pour récupérer une structure DataFrame standard.
`pd.crosstab(index, columns)`	Génère un tableau croisé (matrice de co-occurrences ou de contingence) entre deux variables catégorielles.
`df.apply(fonction, axis=1)`	Applique une fonction de segmentation ou de calcul métier complexe ligne par ligne.